

Numpy

What is NumPy?

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.

NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

NumPy stands for Numerical Python.

Why Use NumPy?

In Python we have lists that serve the purpose of arrays, but they are slow to process.

NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.

The array object in NumPy is called `ndarray`, it provides a lot of supporting functions that make working with `ndarray` very easy.

Example

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

Example

Get the first element from the following array:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```

Example

Access the element on the first row, second column:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0, 1])
```

Negative Indexing

Use negative indexing to access an array from the end.

Example

Print the last element from the 2nd dim:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('Last element from 2nd dim: ', arr[1, -1])
```

Slicing arrays

Slicing in python means taking elements from one given index to another given index.

We pass slice instead of index like this: `[start:end]`.

We can also define the step, like this: `[start:end:step]`.

If we don't pass start its considered 0

If we don't pass end its considered length of array in that dimension

If we don't pass step its considered 1

Example

Slice elements from index 1 to index 5 from the following array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5])
```

Example

Slice elements from index 4 to the end of the array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[4:])
```

Example

Slice elements from the beginning to index 4 (not included):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[:4])
```

Example

Return every other element from index 1 to index 5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

Example

From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[1, 1:4])
```


Example

Print the shape of a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr.shape)
```

Example

Iterate on the elements of the following 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    print(x)
```

Example

Iterate on each scalar element of the 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    for y in x:
        print(y)
```

Example

Join two arrays

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.concatenate((arr1, arr2))

print(arr)
```

Example

Join two 2-D arrays along rows (axis=1):

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```

Access the splitted arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr[0])
print(newarr[1])
print(newarr[2])
```

Find the indexes where the value is 4:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])

x = np.where(arr == 4)

print(x)
```

Sort the array:

```
import numpy as np

arr = np.array([3, 2, 0, 1])

print(np.sort(arr))
```

Add the values in arr1 to the values in arr2:

```
import numpy as np

arr1 = np.array([10, 11, 12, 13, 14, 15])
arr2 = np.array([20, 21, 22, 23, 24, 25])

newarr = np.add(arr1, arr2)

print(newarr)
```

Matrix using Numpy

Create a matrix using Numpy

```
import numpy as np

# 1st argument --> numbers ranging from 0 to 9,
# 2nd argument, row = 3, col = 3
array = np.random.randint(10, size=(3, 3))
print(array)
```

Output:

```
[[2 7 5]
 [8 5 1]
 [8 4 6]]
```

```
# initializing matrices
x = numpy.array([[1, 2], [4, 5]])
y = numpy.array([[7, 8], [9, 10]])

# using add() to add matrices
print ("The element wise addition of matrix is : ")
print (numpy.add(x,y))

# using subtract() to subtract matrices
print ("The element wise subtraction of matrix is : ")
print (numpy.subtract(x,y))

print ("The element wise multiplication of matrix is : ")
print (numpy.multiply(x,y))

# using divide() to divide matrices
print ("The element wise division of matrix is : ")
print (numpy.divide(x,y))
```


Use the NumPy `mean()` method to find the average speed:

```
import numpy

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.mean(speed)

print(x)
```

Use the NumPy `median()` method to find the middle value:

```
import numpy

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.median(speed)
```

Use the SciPy `mode()` method to find the number that appears the most:

```
from scipy import stats

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = stats.mode(speed)

print(x)
```

PANDAS, MATPLOTLIB & SEABORN

What is Pandas?

Pandas is a Python library used for working with data sets.

It has functions for analyzing, cleaning, exploring, and manipulating data.

Example

```
import pandas as pd

mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}

myvar = pd.DataFrame(mydataset)

print(myvar)
```

Pandas use the `loc` attribute to return one or more specified row(s)

Example

Return row 0:

```
#refer to the row index:  
print(df.loc[0])
```

Result

```
calories    420  
duration     50  
Name: 0, dtype: int64
```

Example

Return row 0 and 1:

```
#use a list of indexes:  
print(df.loc[[0, 1]])
```

Load Files Into a DataFrame

If your data sets are stored in a file, Pandas can load them into a DataFrame.

Example

Load a comma separated file (CSV file) into a DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df)
```

Example

Get a quick overview by printing the first 10 rows of the DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

Example

Print the last 5 rows of the DataFrame:

```
print(df.tail())
```

Print information about the data:

```
print(df.info())
```

Result

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Duration    169 non-null    int64
1   Pulse       169 non-null    int64
2   Maxpulse    169 non-null    int64
3   Calories    164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Example

Replace NULL values with the number 130:

Return a new Data Frame with no empty cells:

```
import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()
```

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)
```

Replace NULL values in the "Calories" columns with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df["Calories"].fillna(130, inplace = True)
```

Calculate the MEAN, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Calories"].mean()

df["Calories"].fillna(x, inplace = True)
```

Calculate the MEDIAN, and replace any empty values with it:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Calories"].median()

df["Calories"].fillna(x, inplace = True)
```

Example

Remove all duplicates:

```
df.drop_duplicates(inplace = True)
```


Example

Show the relationship between the columns:

```
df.corr()
```

Result

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922721
Pulse	-0.155408	1.000000	0.786535	0.025120
Maxpulse	0.009403	0.786535	1.000000	0.203814
Calories	0.922721	0.025120	0.203814	1.000000

Note: The `corr()` method ignores "not numeric" columns.

Example

Import pyplot from Matplotlib and visualize our DataFrame:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')

df.plot()

plt.show()
```

What is Matplotlib?

Matplotlib is a low level graph plotting library in python that serves as a visualization utility.

Pyplot

Most of the Matplotlib utilities lies under the `pyplot` submodule, and are usually imported under the `plt` alias:

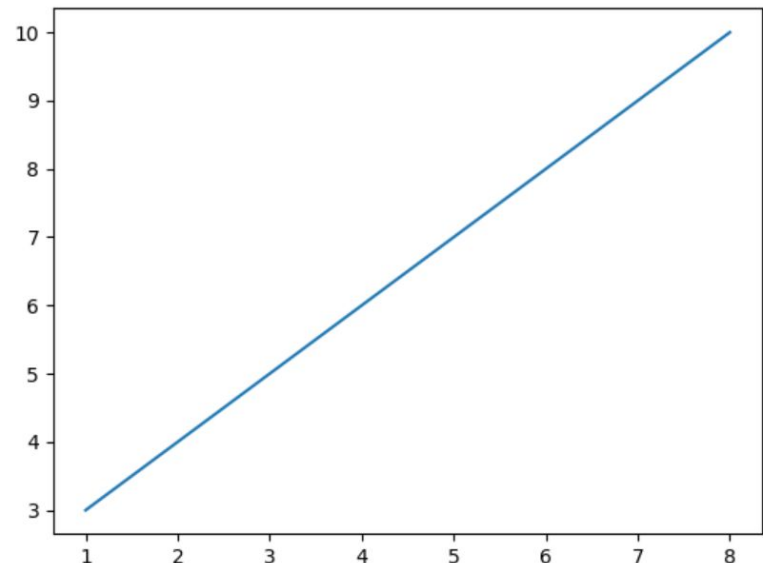
Example

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



A simple scatter plot:

```
import matplotlib.pyplot as plt
```

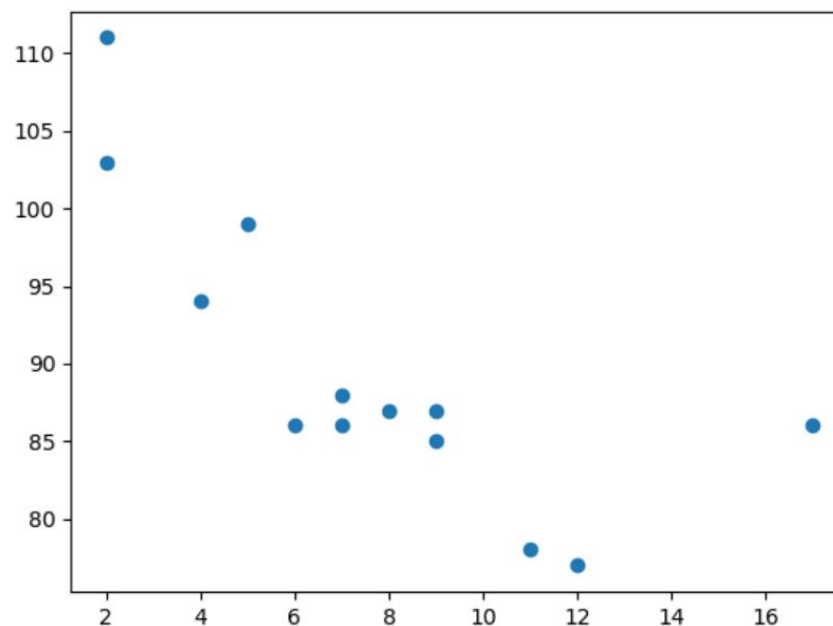
```
import numpy as np
```

```
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
```

```
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
```

```
plt.scatter(x, y)
```

```
plt.show()
```



Visualize Distributions With Seaborn

Seaborn is a library that uses Matplotlib underneath to plot graphs. It will be used to visualize random distributions.

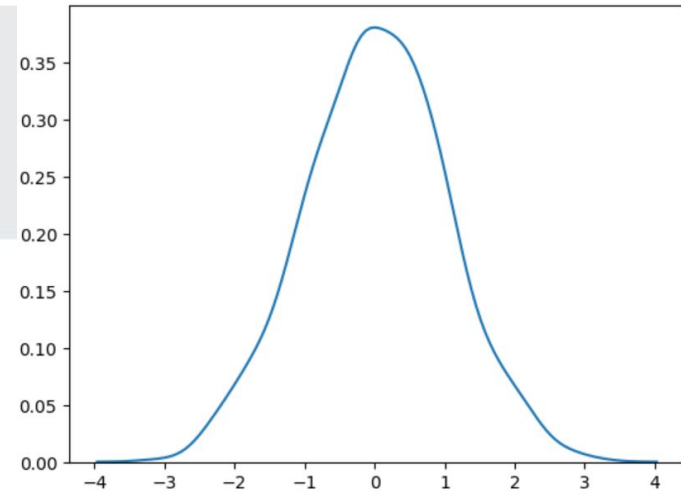
Visualization of Normal Distribution

Example

```
from numpy import random
import matplotlib.pyplot as plt
import seaborn as sns

sns.distplot(random.normal(size=1000), hist=False)

plt.show()
```



```
# importing the modules
import numpy as np
import seaborn as sn
import matplotlib.pyplot as plt

# generating 2-D 10x10 matrix of random numbers
# from 1 to 100
data = np.random.randint(low = 1,
                          high = 100,
                          size = (10, 10))

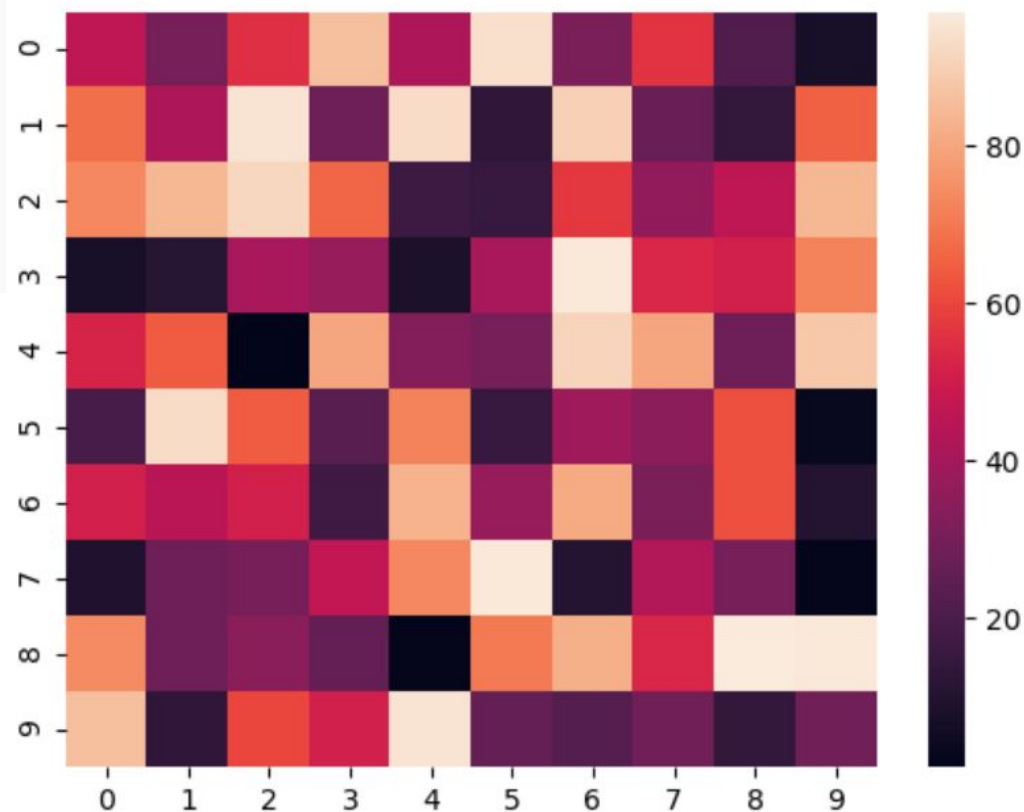
print("The data to be plotted:\n")
print(data)

# plotting the heatmap
hm = sn.heatmap(data = data)

# displaying the plotted heatmap
plt.show()
```

The data to be plotted:

```
[[46 30 55 86 42 94 31 56 21  7]
 [68 42 95 28 93 13 90 27 14 65]
 [73 84 92 66 16 15 57 36 46 84]
 [ 7 11 41 37  8 41 96 53 51 72]
 [52 64  1 80 33 30 91 80 28 88]
 [19 93 64 23 72 15 39 35 62  3]
 [51 45 51 17 83 37 81 31 62 10]
 [ 9 28 30 47 73 96 10 43 30  2]
 [74 28 34 26  2 70 82 53 97 96]
 [86 13 60 51 95 26 22 29 14 29]]
```



Experiment-11: Create NumPy arrays with different methods (lists, zeros, ones, arange).

- Perform mathematical operations on NumPy arrays (addition, subtraction, multiplication, division).
- Practice indexing and slicing NumPy arrays.
- Use NumPy functions for basic statistical analysis like mean, median and mode.
- Experiment with reshaping and transposing NumPy arrays

Experiment-12: Create a Pandas Data Frame from .CSV files (Student can download any CSV file like iris.csv, Indiacrime.csv etc from kaggle)

- Explore basic DataFrame operations, such as head(), tail(), info(), describe().
- Practice selecting and indexing data from a DataFrame.
- Filter data based on conditions.
- Perform basic data cleaning operations, such as handling missing values.
- Practice grouping and aggregating data.
- Use Pandas for basic data visualization

Experiment-13: Perform Data Visualization with Matplotlib and Seaborn.

- Create a simple line plot using Matplotlib.
- Create a scatter plot using Matplotlib.
- Create a bar plot using Seaborn
- Create a histogram using Seaborn.
- Create a box plot using Seaborn.
- Create a heatmap using Seaborn.